

UNIVERSIDADE SÃO JUDAS TADEU

CIÊNCIA DA COMPUTAÇÃO

MATEMÁTICA COMPUTACIONAL APLICADA

GABRIEL ROQUE FRANÇA
VINICIUS PAULO DE ALMEIDA

PRINCÍPIOS DE CONTAGEM E ANÁLISE COMBINATÓRIA EM PROBLEMAS DE
GERAÇÃO DE SENHAS E SEGURANÇA COMPUTACIONAL

PROFESSOR FERNANDO MORI
PROFESSOR ANTONIO TAVARES DE FRANCA JUNIOR

SÃO PAULO

2026

GABRIEL ROQUE FRANÇA
VINICIUS PAULO DE ALMEIDA

PRINCÍPIOS DE CONTAGEM E ANÁLISE COMBINATÓRIA EM PROBLEMAS
DE GERAÇÃO DE SENHAS E SEGURANÇA COMPUTACIONAL

TRABALHO APRESENTADO À DISCIPLINA DE MATEMÁTICA
COMPUTACIONAL APLICADA DO CURSO DE CIÊNCIA DA COMPUTAÇÃO,
COMO REQUISITO PARCIAL PARA AVALIAÇÃO

SÃO PAULO
2026

SUMÁRIO

1 INTRODUÇÃO.....	1
2 FUNDAMENTAÇÃO TEÓRICA.....	3
2.1 Princípio Fundamental da Contagem.....	3
2.2 Permutação.....	4
2.3 Arranjo.....	4
2.4 Combinação.....	5
3 APLICAÇÕES EM COMPUTAÇÃO.....	6
3.1 Espaço de Busca.....	6
3.2 Crescimento Exponencial.....	8
3.3 Ataque por força bruta.....	9
3.4 Aplicações reais.....	10
4 ESTUDO DE CASO: GERADOR E ANALISADOR DE SENHAS EM PYTHON.....	11
4.1 Objetivo do estudo de caso.....	11
4.2 Funcionamento do algoritmo.....	11
4.3 Base matemática.....	12
4.4 Exemplo de entrada e resultado.....	13
4.5 Código completo em Python.....	14
4.6 Análise: combinatória aplicada na prática.....	16
4.7 Conexão com ataque por força bruta, crescimento exponencial e segurança digital.....	16
5 CONCLUSÃO.....	18
REFERÊNCIAS.....	20

1 INTRODUÇÃO

Nos últimos anos, o uso da internet cresceu de forma exponencial. No Brasil, segundo o CERT.br (Centro de Estudos, Resposta e Tratamento de Incidentes de Segurança no Brasil), foram registrados mais de 2,3 milhões de incidentes de segurança apenas em 2023 — número que vem aumentando a cada ano. Esse dado mostra que proteger informações digitais deixou de ser uma preocupação exclusiva de especialistas e passou a fazer parte do cotidiano de qualquer pessoa que utiliza um smartphone, acessa um banco pelo aplicativo ou mantém uma conta em rede social.

A senha é, ainda hoje, a forma mais comum de proteger o acesso a esses sistemas. Mas nem toda senha oferece o mesmo nível de proteção. Uma senha simples, como "1234" ou o próprio nome do usuário, pode ser descoberta em questão de segundos por um programa automatizado. Já uma senha longa, com letras maiúsculas, minúsculas, números e caracteres especiais, pode levar anos para ser quebrada — e essa diferença tem uma explicação matemática precisa.

E é justamente aí que a matemática entra. Um dos ataques mais conhecidos na área de segurança digital é o chamado ataque por força bruta: o sistema tenta, uma por uma, todas as combinações possíveis até encontrar a senha certa. Quanto mais combinações existirem, mais tempo o ataque leva. Esse conjunto de possibilidades tem um nome técnico: espaço de busca. É fundamental saber calcular esse espaço para entender por que algumas senhas são seguras e outras não.

O cálculo do espaço de busca é feito utilizando os princípios da análise combinatória. Como afirmam Iezzi et al. (2013, p. 1), "a análise combinatória tem por objetivo determinar o número de maneiras pelas quais se podem formar agrupamentos de elementos". Aplicando esses princípios à geração de senhas, é possível calcular exatamente quantas senhas diferentes podem ser formadas a partir de um determinado conjunto de caracteres — e o resultado, muitas vezes, surpreende pela dimensão dos números envolvidos.

Portanto, a matemática e a computação, não são áreas separadas. Os algoritmos que protegem transações bancárias, geram tokens de autenticação e sustentam sistemas de criptografia

são construídos sobre fundamentos e bases matemáticas. Entender esses fundamentos ajuda a tomar decisões mais conscientes sobre segurança digital — tanto no desenvolvimento de sistemas quanto no uso do dia a dia da tecnologia.

Com base nisso, este trabalho busca mostrar como os conceitos da análise combinatória se aplicam na formação de senhas e na avaliação da segurança computacional. Serão apresentados o Princípio Fundamental da Contagem, as permutações, os arranjos e as combinações, sempre relacionando cada conceito a situações reais da computação. Ao final, um estudo de caso comparará diferentes tipos de senhas, demonstrando, com cálculos concretos, de que forma a escolha dos caracteres e o comprimento da senha afetam diretamente a proteção do sistema.

2 FUNDAMENTAÇÃO TEÓRICA

Para compreender de que forma a matemática se aplica à segurança de senhas, é necessário conhecer alguns conceitos centrais da análise combinatória. Essa área da matemática estuda as diferentes maneiras de organizar e selecionar elementos de um conjunto, fornecendo ferramentas precisas para calcular o número total de possibilidades em diferentes situações.

2.1 Princípio Fundamental da Contagem

O Princípio Fundamental da Contagem (PFC), também chamado de regra do produto, é a base de toda a análise combinatória. Ele estabelece que, se uma tarefa pode ser realizada de m maneiras distintas e, para cada uma delas, uma segunda tarefa puder ser realizada de n maneiras distintas, então o número total de formas de realizar as duas tarefas é igual ao produto $m \times n$. Esse raciocínio se estende para qualquer quantidade de tarefas encadeadas.

Aplicando esse princípio à formação de senhas: se cada posição da senha pode ser preenchida com qualquer caractere de um alfabeto de c símbolos, e a senha possui n posições, então o total de senhas possíveis é:

$$E = c^n$$

Esse resultado é o que chamamos de espaço de busca — o número exato de combinações que um atacante precisaria testar no pior caso. Uma senha de quatro dígitos numéricos, por exemplo, apresenta $10^4 = 10.000$ possibilidades. Uma senha de oito caracteres com letras maiúsculas, minúsculas e números (62 opções) eleva esse total para $62^8 \approx 218$ bilhões de combinações — uma diferença de mais de vinte milhões de vezes.

2.2 Permutação

A permutação estuda o número de formas distintas de ordenar todos os elementos de um conjunto. Quando se reorganizam n elementos distintos, o número total de arranjos possíveis é chamado de fatorial de n , representado por $n!$, e calculado como:

$$P_n = n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

Por exemplo, o conjunto de letras $\{A, B, C\}$ pode ser organizado de $3! = 3 \times 2 \times 1 = 6$ maneiras distintas: ABC, ACB, BAC, BCA, CAB e CBA. Em todos os casos, todos os elementos são utilizados e a ordem importa.

Na segurança computacional, a permutação aparece em contextos onde os caracteres não se repetem e todos os elementos disponíveis são utilizados, como no embaralhamento de chaves em determinados algoritmos criptográficos. No entanto, como a maioria das senhas permite repetição de caracteres, a permutação pura é mais rara nesse contexto e serve principalmente como fundamento para os conceitos de arranjo e combinação.

2.3 Arranjo

O arranjo é uma generalização da permutação para situações em que apenas uma parte dos elementos do conjunto é selecionada e organizada. Dado um conjunto de n elementos distintos, o arranjo de p elementos (com $p \leq n$) representa o número de formas de selecionar e ordenar p desses elementos. A fórmula é:

$$A_{n,p} = n! / (n - p)!$$

Como exemplo, considere a pergunta: de quantas formas é possível formar senhas de 4 letras distintas usando o alfabeto de 26 letras? A resposta é $A_{26,4} = 26! / (26 - 4)! = 26 \times 25 \times 24 \times 23 =$

358.800 senhas distintas. A diferença em relação à permutação está no fato de que apenas 4 dos 26 elementos são utilizados, e a ordem ainda importa.

Vale destacar que o arranjo pressupõe ausência de repetição: cada caractere só pode aparecer uma vez na senha. Como a maioria dos sistemas permite repetição, o arranjo simples é menos comum na prática do que a fórmula do espaço de busca derivada do PFC. Ainda assim, é um conceito importante para análises de protocolos que impõem restrições de unicidade de caracteres.

2.4 Combinação

A combinação trata da seleção de p elementos de um conjunto de n elementos distintos, sem considerar a ordem em que aparecem. Diferentemente do arranjo, em que ABC e BAC são resultados distintos, na combinação eles são considerados equivalentes. A fórmula é:

$$C_{n,p} = n! / (p! \times (n - p)!)$$

Um exemplo prático: de quantas formas é possível escolher 3 tipos de caracteres especiais de um conjunto de 10 disponíveis para incluir em uma política de senha? A resposta é $C_{10,3} = 10! / (3! \times 7!) = 120$ combinações possíveis.

Na segurança computacional, a combinação é aplicada em análises de protocolos criptográficos que trabalham com subconjuntos de parâmetros ou chaves, e também em estudos de probabilidade ligados a ataques de dicionário, nos quais a ordem dos caracteres é irrelevante para determinar se uma senha pertence a uma lista de palavras comuns. Combinada com os demais conceitos, ela completa o ferramental matemático necessário para avaliar a robustez de sistemas de autenticação.

3 APLICAÇÕES EM COMPUTAÇÃO

A análise combinatória não é apenas teoria. Ela aparece de forma central em sistemas computacionais reais, naqueles voltados aos princípios de segurança digital. Nas seções a seguir, serão apresentadas as aplicações práticas de cada um dos conceitos matemáticos estudados anteriormente — começando pelo mais fundamental de todos: o espaço de busca.

3.1 Espaço de Busca

Para que possamos entender do por que algumas senhas são mais seguras do que outras, é necessário primeiro compreender um conceito fundamental: o espaço de busca. Em termos simples, o espaço de busca é o conjunto de todas as senhas possíveis que podem ser formadas a partir de um determinado conjunto de regras. Se um sistema permite senhas com quatro dígitos numéricos, por exemplo, o espaço de busca inclui desde “0000” até “9999” — um total de 10.000 combinações.

Esse número é muito importante, pois define o custo de um ataque. Em um ataque por força bruta, o invasor testa sistematicamente todas as combinações possíveis. Se o espaço de busca tem 10.000 possíveis senhas, o ataque pode ser concluído rapidamente. Se tem trilhões, pode levar anos — mesmo o invasor tendo computadores potentes. A segurança, portanto, não depende só de “ter uma senha difícil de adivinhar”; depende de existirem combinações suficientes para tornar o teste exaustivo inviável.

O tamanho do espaço de busca é calculado através do Princípio Fundamental da Contagem. Para uma senha de comprimento n formada por caracteres retirados de um alfabeto de tamanho c , o espaço de busca é dado por:

$$E = c^n$$

Essa fórmula aparentemente simples pode trazer consequências enormes. Pequenas mudanças nos valores de c ou n podem provocar crescimento exponencial no espaço de busca — e é exatamente isso que torna senhas mais longas ou com mais tipos de caracteres tão mais seguras.

Para entender na prática, é necessário comparar alguns casos. Uma senha formada apenas por dígitos numéricos ($c = 10$) e com quatro caracteres ($n = 4$) resulta em $10^4 = 10.000$ possibilidades. Já uma senha com oito caracteres ($n = 8$) e misturando letras maiúsculas, minúsculas e números ($c = 62$) resulta em 62^8 , o que equivale a aproximadamente 218 bilhões de combinações. E se símbolos especiais forem incluídos, ampliando o alfabeto para 95 caracteres, o espaço de busca sobe para 95^8 — resultando em aproximadamente cerca de 6,6 trilhões de possibilidades.

Tabela 1 — Comparação do espaço de busca por tipo de senha

Tipo de senha	Espaço de busca	Nível de segurança
Apenas números (4 dígitos) ex: 1234	$10^4 = 10.000$	Muito baixo
Apenas números (6 dígitos) ex: 482031	$10^6 = 1.000.000$	Baixo
Letras minúsculas (6 car.) ex: abcdef	$26^6 \approx 308$ milhões	Médio
Letras + números (8 car.) ex: abc12345	$62^8 \approx 218$ bilhões	Alto
Letras + números + símbolos (8 car.) ex: G@b2026!	$95^8 \approx 6,6$ trilhões	Muito alto

Fonte: elaborada pelos autores (2026).

Os dados da tabela tornam visível o que a fórmula apenas sugere: o crescimento é exponencial, não linear. Passar de quatro para seis dígitos numéricos multiplica o espaço de busca

por 100. Introduzir letras o multiplica por milhares. Acrescentar símbolos vai além dos trilhões. Cada camada de complexidade adicionada não soma possibilidades — ela as multiplica.

Por isso, sistemas computacionais modernos costumam exigir senhas com requisitos mínimos de complexidade: comprimento de ao menos oito caracteres, uso de letras maiúsculas e minúsculas, números e símbolos. Essas exigências não são arbitrárias. Elas têm fundamento matemático direto: cada requisito amplia o valor de c ou de n na fórmula do espaço de busca, tornando o ataque por força bruta progressivamente mais custoso — e, na prática, inviável dentro de qualquer janela de tempo razoável.

3.2 Crescimento Exponencial

Quando se fala que senhas mais complexas são mais seguras, o que está por trás dessa afirmação é o crescimento exponencial do espaço de busca. Esse tipo de crescimento é diferente do crescimento linear — e a diferença é enorme. No crescimento linear, cada acréscimo soma uma quantidade fixa. No crescimento exponencial, cada acréscimo multiplica o valor anterior. No caso das senhas, adicionar um único caractere ao comprimento n não aumenta o espaço de busca em c unidades: ele o multiplica por c .

Um exemplo torna isso concreto. Considere senhas formadas apenas por letras minúsculas do alfabeto ($c = 26$). Uma senha de 4 caracteres produz $26^4 = 456.976$ possibilidades. Ao passar para 5 caracteres, o resultado sobe para $26^5 = 11.881.376$ — um aumento de quase 26 vezes. Com 6 caracteres, chega-se a $26^6 \approx 308$ milhões. Com 8 caracteres, a marca ultrapassa 200 bilhões. Cada caractere a mais não representa um passo — representa um salto.

O mesmo efeito ocorre quando se amplia o alfabeto. Manter o comprimento em 8 caracteres, mas incluir letras maiúsculas junto com as minúsculas ($c = 52$), eleva o espaço de busca para aproximadamente $52^8 \approx 53$ bilhões. Acrescentar os dígitos de 0 a 9 ($c = 62$) leva a $62^8 \approx 218$ bilhões. Incluir símbolos especiais e chegar a $c = 95$ resulta em $95^8 \approx 6,6$ trilhões. Em todos os casos, o padrão é o mesmo: a função cresce de forma não linear, e cada expansão do alfabeto ou do comprimento impacta o resultado de maneira desproporcional.

Essa propriedade do crescimento exponencial é o que torna a análise combinatória tão relevante para a segurança digital. Não basta apenas aumentar o comprimento da senha ou incluir mais tipos de caracteres sem entender o impacto matemático dessas escolhas. Com a fórmula $E = c^n$, é possível calcular com precisão o efeito de cada decisão de design sobre a robustez do sistema.

3.3 Ataque por força bruta

O ataque por força bruta é uma das técnicas mais antigas e diretas da segurança computacional. O princípio é simples: o atacante utiliza um programa que gera e testa automaticamente todas as combinações possíveis de uma senha, uma a uma, até encontrar a correta. Não há estratégia ou adivinhação — apenas exaustão sistemática do espaço de busca.

A viabilidade do ataque depende diretamente do tamanho do espaço de busca e da velocidade de processamento do computador utilizado. Para ter uma noção concreta: um computador pessoal comum consegue testar em torno de 1 bilhão de combinações por segundo em cenários simples. Isso significa que uma senha numérica de 4 dígitos (10.000 possibilidades) seria quebrada em menos de um milissegundo. Já uma senha com 8 caracteres misturando letras, números e símbolos ($95^8 \approx 6,6$ trilhões) levaria cerca de 76 dias no mesmo equipamento — e anos em sistemas com proteções adicionais de limite de tentativas.

É importante destacar que a relação entre o ataque por força bruta e a combinatória não é apenas teórica. O número máximo de tentativas necessárias para garantir que a senha será encontrada é exatamente o espaço de busca $E = c^n$. Na prática, a senha correta pode ser encontrada antes de esgotar todas as possibilidades — mas, no pior caso, o atacante precisará testar cada uma delas. Por isso, sistemas bem projetados não apenas exigem senhas complexas, mas também implementam mecanismos como bloqueio após múltiplas tentativas incorretas, tempo de espera entre tentativas e autenticação em dois fatores — todos eles voltados a tornar o ataque por força bruta ainda mais custoso.

3.4 Aplicações reais

Os conceitos apresentados até aqui não ficam restritos ao ambiente acadêmico. Eles estão presentes em sistemas que qualquer pessoa utiliza no dia a dia, muitas vezes sem perceber. A seguir, são apresentadas algumas das principais aplicações reais da combinatória na computação.

Autenticação de usuários. Todo sistema que exige login — e-mail, redes sociais, plataformas de streaming, aplicativos bancários — depende de um mecanismo de autenticação baseado em senhas. As regras de complexidade que esses sistemas impõem ao criar uma senha (tamanho mínimo, uso de maiúsculas, números, símbolos) são diretamente derivadas do cálculo do espaço de busca. Quanto mais restritivas as regras, maior o espaço de busca resultante e mais resistente o sistema a ataques.

Geração de tokens e códigos OTP. Quando um banco envia um código de 6 dígitos por SMS para confirmar uma transferência, está utilizando um OTP (One-Time Password) — uma senha de uso único. Esse código é gerado aleatoriamente dentro de um espaço de busca de $10^6 = 1.000.000$ possibilidades. A validade curta (geralmente 30 ou 60 segundos) combinada com o limite de tentativas transforma esse espaço em uma barreira prática extremamente eficaz contra interceptação.

Criptografia e chaves digitais. Na criptografia, o princípio é o mesmo, mas em escala muito maior. Uma chave criptográfica de 128 bits, padrão utilizado pelo protocolo AES (Advanced Encryption Standard), possui um espaço de busca de 2^{128} — um número com 39 dígitos. Mesmo com supercomputadores testando bilhões de combinações por segundo, seria humanamente impossível percorrer todo esse espaço dentro de qualquer prazo razoável. A segurança do sistema, portanto, não depende de segredo absoluto — depende da magnitude matemática do espaço de busca.

Sistemas bancários e PIN. O PIN de um cartão bancário tem apenas 4 dígitos numéricos — espaço de busca de apenas 10.000 combinações. Por que esse sistema ainda é considerado seguro? Porque a segurança não está no tamanho do espaço de busca, mas no bloqueio automático após três tentativas incorretas. Esse mecanismo torna o ataque por força bruta inviável na prática — mesmo

com um espaço pequeno. É um exemplo de como a segurança real combina matemática com design de sistema.

Em todos esses casos, o fio condutor é o mesmo: a análise combinatória fornece a base matemática para medir, projetar e avaliar a segurança de sistemas digitais. Saber calcular o espaço de busca é saber calcular o custo de um ataque — e, portanto, saber o quanto um sistema é resistente.

4 ESTUDO DE CASO: GERADOR E ANALISADOR DE SENHAS EM PYTHON

4.1 Objetivo do estudo de caso

As seções anteriores apresentaram os fundamentos teóricos da análise combinatória e suas conexões com a segurança computacional. Este estudo de caso tem como objetivo demonstrar, de forma prática, como esses conceitos se traduzem em código real. Para isso, foi desenvolvido um algoritmo em Python que recebe configurações de senha definidas pelo usuário, calcula matematicamente o espaço de busca correspondente, gera três sugestões de senha e classifica o nível de segurança do sistema.

A escolha do Python como linguagem se justifica por sua sintaxe clara e pela sua facilidade de leitura do código, características que tornam o algoritmo adequado para fins didáticos. O programa não utiliza bibliotecas externas além das que acompanham a instalação padrão da linguagem, o que garante que qualquer pessoa possa executá-lo sem configuração adicional.

4.2 Funcionamento do algoritmo

O algoritmo inicia solicitando ao usuário duas informações fundamentais: o tamanho desejado da senha e os tipos de caracteres que devem compô-la. Para esta segunda informação, o

programa apresenta quatro opções independentes, cada uma correspondendo a um subconjunto do alfabeto disponível:

- a) **letras minúscula** — as 26 letras do alfabeto latino em sua forma minúsculas (a-z), totalizando 26 caracteres;
- b) **letras maiúscula** — as mesmas 26 letras em forma maiúsculas (A-Z), acrescentando outros 26 caracteres ao alfabeto;
- c) **números** — os dígitos de 0 a 9, que contribuem com 10 caracteres adicionais;
- d) **símbolos** — caracteres especiais como !, @, #, \$, %, entre outros, totalizando 32 possibilidades.

Cada opção marcada pelo usuário amplia o conjunto de caracteres disponíveis para a construção da senha. O tamanho total deste conjunto — denominado alfabeto — define o valor de c na fórmula do espaço de busca. O tamanho da senha, por sua vez, define o valor de n . Com esses dois parâmetros, o algoritmo aplica diretamente o Princípio Fundamental da Contagem para calcular o espaço de busca total.

4.3 Base matemática

O cálculo central do algoritmo é baseado na fórmula do espaço de busca, derivada diretamente do Princípio Fundamental da Contagem:

$$E = c^n$$

onde E representa o espaço de busca, isto é, o número total de senhas distintas que podem ser formadas; c representa a quantidade de caracteres disponíveis no alfabeto; e n representa o comprimento da senha, ou seja, o número de posições a serem preenchidas.

A lógica por trás da fórmula é direta: cada posição da senha pode ser preenchida com qualquer um dos c caracteres disponíveis, de forma independente. Pelo Princípio Fundamental da Contagem, o número total de possibilidades é obtido multiplicando-se as opções de cada posição —

ou seja, $c \times c \times c \dots n$ vezes, o que resulta em c^n . Em Python, essa operação é representada pela expressão `espaco_busca = c ** n`, onde o operador `**` realiza a exponenciação.

4.4 Exemplo de entrada e resultado

Para ilustrar o funcionamento do algoritmo, considere um usuário que define uma senha de 8 caracteres utilizando letras minúsculas, maiúsculas e números, mas sem símbolos. A tabela a seguir apresenta os parâmetros informados e os resultados calculados pelo programa.

Tabela 2 — Exemplo de entrada e resultado do algoritmo

Parâmetros de entrada	Valor informado	Resultado calculado
Tamanho da senha	8 caracteres	$n = 8$
Letras minúsculas	Sim	+26 caracteres
Letras maiúsculas	Sim	+26 caracteres
Números	Sim	+10 caracteres
Símbolos	Não	+0 caracteres
Alfabeto total (c)	62 caracteres	$c = 62$
Espaço de busca (E)	62^8	$\approx 218.340.105.584.896$
Nível de segurança	—	Muito alto

Fonte: elaborada pelos autores (2026).

Com $c = 62$ e $n = 8$, o espaço de busca calculado é $62^8 = 218.340.105.584.896$ — mais de 218 trilhões de combinações. Esse valor supera os limiares definidos no algoritmo para os níveis “Alto” e “Muito alto”, levando o programa a classificar a senha como de segurança muito alta. Para

um computador testando 1 bilhão de combinações por segundo — velocidade realista para ataques comuns —, percorrer todo esse espaço levaria mais de 2.500 dias.

4.5 Código completo em Python

O código a seguir representa a implementação completa do algoritmo descrito. Cada bloco é acompanhado de comentários que explicam sua função dentro do programa:

```
# =====
# Gerador e Analisador de Senhas
# Trabalho Acadêmico - Matemática Computacional Aplicada
# =====

import random
import string

# --- Configurações do usuário ---
print('=== Gerador e Analisador de Senhas ===')
print()

# Tamanho da senha
tamanho = int(input('Informe o tamanho desejado da senha: '))

# Escolha dos tipos de caracteres
usar_minusculas = input('Usar letras minúsculas? (s/n):').strip().lower() == 's'
usar_maiusculas = input('Usar letras maiúsculas? (s/n):').strip().lower() == 's'
usar_numeros = input('Usar números? (s/n):').strip().lower() == 's'
usar_simbolos = input('Usar símbolos? (s/n):').strip().lower() == 's'

# --- Montagem do alfabeto disponível ---
alfabeto = ''

if usar_minusculas:
    alfabeto += string.ascii_lowercase # a-z (26 caracteres)

if usar_maiusculas:
    alfabeto += string.ascii_uppercase # A-Z (26 caracteres)

if usar_numeros:
```

```

alfabeto += string.digits          # 0-9 (10 caracteres)

if usar_simbolos:
    alfabeto += string.punctuation # !@#$... (32 caracteres)

# Verificação: ao menos um tipo deve ser escolhido
if not alfabeto:
    print('Erro: selecione ao menos um tipo de caractere.')
    exit()

# --- Cálculo do espaço de busca ( $E = c^n$ ) ---
c = len(alfabeto)      # tamanho do alfabeto
n = tamanho            # comprimento da senha
espaco_busca = c ** n  # Princípio Fundamental da Contagem

# --- Classificação do nível de segurança ---
if espaco_busca < 1_000_000:
    nivel = 'Muito baixo'
elif espaco_busca < 1_000_000_000:
    nivel = 'Baixo'
elif espaco_busca < 1_000_000_000_000:
    nivel = 'Médio'
elif espaco_busca < 1_000_000_000_000_000:
    nivel = 'Alto'
else:
    nivel = 'Muito alto'

# --- Geração de 3 sugestões de senha ---
print()
print('--- Sugestões de senha ---')
for i in range(1, 4):
    senha = ''.join(random.choices(alfabeto, k=tamanho))
    print(f'  Sugestão {i}: {senha}')

# --- Exibição dos resultados ---
print()
print('--- Análise do espaço de busca ---')
print(f'  Alfabeto utilizado : {c} caracteres')
print(f'  Tamanho da senha   : {n} caracteres')
print(f'  Espaço de busca      : {espaco_busca:,} combinações')
print(f'  Nível de segurança   : {nivel}')

```

Fonte: elaborada pelos autores (2026).

4.6 Análise: combinatória aplicada na prática

O algoritmo desenvolvido é, em sua essência, uma implementação direta da análise combinatória. Cada linha de código que monta o alfabeto corresponde a uma etapa do Princípio Fundamental da Contagem: ao somar os subconjuntos de caracteres escolhidos pelo usuário, o programa está definindo o valor de c . Ao receber o tamanho da senha, está definindo o valor de n . E ao calcular $c ** n$, está aplicando a fórmula $E = c^n$ de forma computacional.

Mais do que calcular um número, o algoritmo torna visível a relação entre as escolhas do usuário e a segurança resultante. Um usuário que opta apenas por números e define uma senha curta verá um espaço de busca pequeno e uma classificação de segurança baixa. Outro que inclui todos os tipos de caracteres e escolhe um comprimento maior terá um resultado radicalmente diferente — e poderá compreender, com base nos números exibidos, o motivo dessa diferença. Essa capacidade de quantificar a segurança é o que distingue uma análise fundamentada de uma percepção intuitiva.

O programa também demonstra algo relevante sobre a natureza da combinatória na computação: a matemática não apenas descreve o problema, ela o resolve. A mesma fórmula utilizada para explicar teoricamente por que senhas complexas são mais seguras é a que o algoritmo utiliza para calcular, de forma precisa, o quão segura é cada configuração. Não há separação entre teoria e prática — são a mesma coisa expressa em linguagens diferentes.

4.7 Conexão com ataque por força bruta, crescimento exponencial e segurança digital

O estudo de caso reúne, em um único exemplo concreto, os três eixos centrais do trabalho. O primeiro é o ataque por força bruta: o espaço de busca calculado pelo algoritmo é exatamente o número máximo de tentativas que um atacante precisaria fazer para garantir a descoberta da senha. Quanto maior esse número, mais tempo e recursos computacionais o ataque exige — e mais seguro o sistema se torna.

O segundo eixo é o crescimento exponencial. Basta testar o algoritmo com diferentes configurações para perceber que pequenas variações nos parâmetros produzem diferenças enormes no espaço de busca. Aumentar a senha de 6 para 8 caracteres, mantendo $c = 62$, eleva o espaço de busca de $62^6 \approx 56$ bilhões para $62^8 \approx 218$ bilhões — um aumento de quase 4.000 vezes. Incluir símbolos e passar de $c = 62$ para $c = 94$ em uma senha de 8 caracteres eleva o resultado para $94^8 \approx 6$ quadrilhões. Esse comportamento é o crescimento exponencial em ação, e o algoritmo o torna observável de forma direta.

O terceiro eixo é a segurança digital. As classificações exibidas pelo programa — de “Muito baixo” a “Muito alto” — traduzem os números matemáticos em orientações práticas.

Elas refletem critérios reais utilizados por sistemas de segurança para avaliar a robustez de senhas e orientar os usuários na criação de credenciais mais resistentes. Ao combinar geração, cálculo e classificação em um único fluxo, o algoritmo demonstra como a matemática se integra ao ciclo completo de um sistema de autenticação seguro.

Em síntese, o estudo de caso confirma a tese central do trabalho: a segurança digital depende diretamente da quantidade de combinações possíveis em um sistema, e a análise combinatória é a ferramenta matemática que permite calcular, compreender e projetar essa segurança com precisão.

5 CONCLUSÃO

Este trabalho partiu de uma questão aparentemente simples: por que algumas senhas são mais seguras do que outras? A resposta, como se demonstrou ao longo das seções, não está em intuição ou convenção — está na matemática. Mais especificamente, está na análise combinatória e na capacidade que ela oferece de calcular, com precisão, o número total de possibilidades existentes em um sistema.

A fundamentação teórica apresentou os quatro conceitos centrais da análise combinatória: o Princípio Fundamental da Contagem, a permutação, o arranjo e a combinação. Cada um desses conceitos descreve uma forma diferente de organizar ou selecionar elementos de um conjunto — e todos eles encontram aplicação direta na computação. O Princípio Fundamental da Contagem, em especial, mostrou-se o alicerce do trabalho inteiro: é dele que deriva a fórmula $E = c^n$, que quantifica o espaço de busca de uma senha a partir do tamanho do alfabeto utilizado e do comprimento da sequência.

As aplicações em computação demonstraram que esses conceitos não são abstrações teóricas. O espaço de busca, o crescimento exponencial, o ataque por força bruta e os sistemas reais de autenticação são todos governados pelos mesmos princípios matemáticos estudados na fundamentação. O crescimento exponencial, em particular, revelou-se uma propriedade de consequências práticas enormes: adicionar um único caractere ao comprimento de uma senha ou incluir um novo conjunto de símbolos não produz um aumento proporcional no espaço de busca — produz um salto de magnitude. Essa propriedade é o que torna a análise combinatória indispensável para o design de sistemas seguros.

O estudo de caso consolidou essa relação entre teoria e prática. O algoritmo desenvolvido em Python demonstrou, em código funcional, que a mesma fórmula usada para explicar a segurança de senhas é a que um programa real utiliza para calculá-la. Ao solicitar os parâmetros ao usuário, montar o alfabeto, aplicar a exponenciação e classificar o resultado, o algoritmo percorre exatamente o mesmo raciocínio matemático apresentado na fundamentação teórica — só que expresso em uma linguagem de programação. Não há separação entre a matemática e o código: são representações diferentes do mesmo problema.

Os três eixos que atravessaram o trabalho — **espaço de busca**, **crescimento exponencial** e **ataque por força bruta** — mostraram-se profundamente interligados. O espaço de busca define o problema; o crescimento exponencial explica por que pequenas mudanças na senha têm impacto tão grande na segurança; e o ataque por força bruta é o cenário concreto em que esse impacto se manifesta. Juntos, eles formam uma estrutura coerente que conecta a matemática à realidade dos sistemas computacionais.

Do ponto de vista prático, o trabalho evidenciou que decisões cotidianas — como escolher uma senha de oito caracteres com letras, números e símbolos em vez de um PIN de quatro dígitos — têm fundamento matemático preciso. A diferença entre 10.000 e 218 trilhões de combinações não é uma estimativa: é o resultado direto da fórmula $E = c^n$ aplicada a parâmetros reais. Compreender esse fundamento permite tanto ao desenvolvedor de sistemas quanto ao usuário comum tomar decisões mais conscientes e fundamentadas sobre segurança digital.

Em última análise, o trabalho confirma que a segurança digital é, em grande medida, um problema matemático. Proteger um sistema não significa torná-lo impenetrável — significa tornar o custo de um ataque suficientemente alto para que ele se torne inviável. E é a análise combinatória que fornece as ferramentas para calcular, projetar e avaliar esse custo com rigor. A matemática, nesse sentido, não é apenas uma disciplina de apoio à computação: ela é parte constitutiva de qualquer sistema digital seguro.

REFERÊNCIAS

CENTRO DE ESTUDOS, RESPOSTA E TRATAMENTO DE INCIDENTES DE SEGURANÇA NO BRASIL (CERT.br). **Estatísticas dos incidentes reportados ao CERT.br**. Núcleo de Informação e Coordenação do Ponto BR (NIC.br). Disponível em: <https://cert.br/stats/>. Acesso em: mai. 2026.

IEZZI, Gelson; HAZZAN, Samuel. **Fundamentos de Matemática Elementar**: combinatória e probabilidade. 8. ed. São Paulo: Atual, 2013. v. 5. ISBN 978-85-357-1750-1.

KEMP, Simon. **Digital 2024: Global Overview Report**. DataReportal; We Are Social; Meltwater, jan. 2024. Disponível em: <https://datareportal.com/reports/digital-2024-global-overview-report>. Acesso em: mai. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **FIPS PUB 197: Advanced Encryption Standard (AES)**. Gaithersburg: U.S. Department of Commerce, 2001. Disponível em: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>. Acesso em: mai. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3 Documentation: library — random**. Disponível em: <https://docs.python.org/3/library/random.html>. Acesso em: mai. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3 Documentation: library — string**. Disponível em: <https://docs.python.org/3/library/string.html>. Acesso em: mai. 2026.

NIST. FIPS PUB 197: **Advanced Encryption Standard (AES)**.